Buổi 5 : Data Modeling và Resource Design

Nhóm 1

- 23020094 Tôn THiện Khỏe
- 21020123 Nguyễn Tiến Hoàng
- 2302014 Hà Vũ Công
- 22024538 Trần Hữu Mạnh

Kiến thức cần đạt

- Thiết kế resource tree phù hợp với domain (ví dụ: /users/{id}/orders)
- Hiểu các chiến lược phân trang (pagination): offset/limit, cursor, page-based

Thực hành

- Dự án nhóm: Thiết kế data model cho hệ thống quản lý thư viện
- Thiết kế endpoint cho tìm kiếm và phân trang

Data Modeling

Data Modeling là gì ?

Định nghĩa : Data Modeling là quá trình thiết kế cách dữ liệu được tổ chức, lưu trữ và liên kết với nhau trong hệ thống

Thành phần Data modeling

Entity (Thực thể) : là một đối tượng hoặc khái niệm trong hệ thống mà ta cần lưu trữ thông tin về nó trong cơ sở dữ liệu.

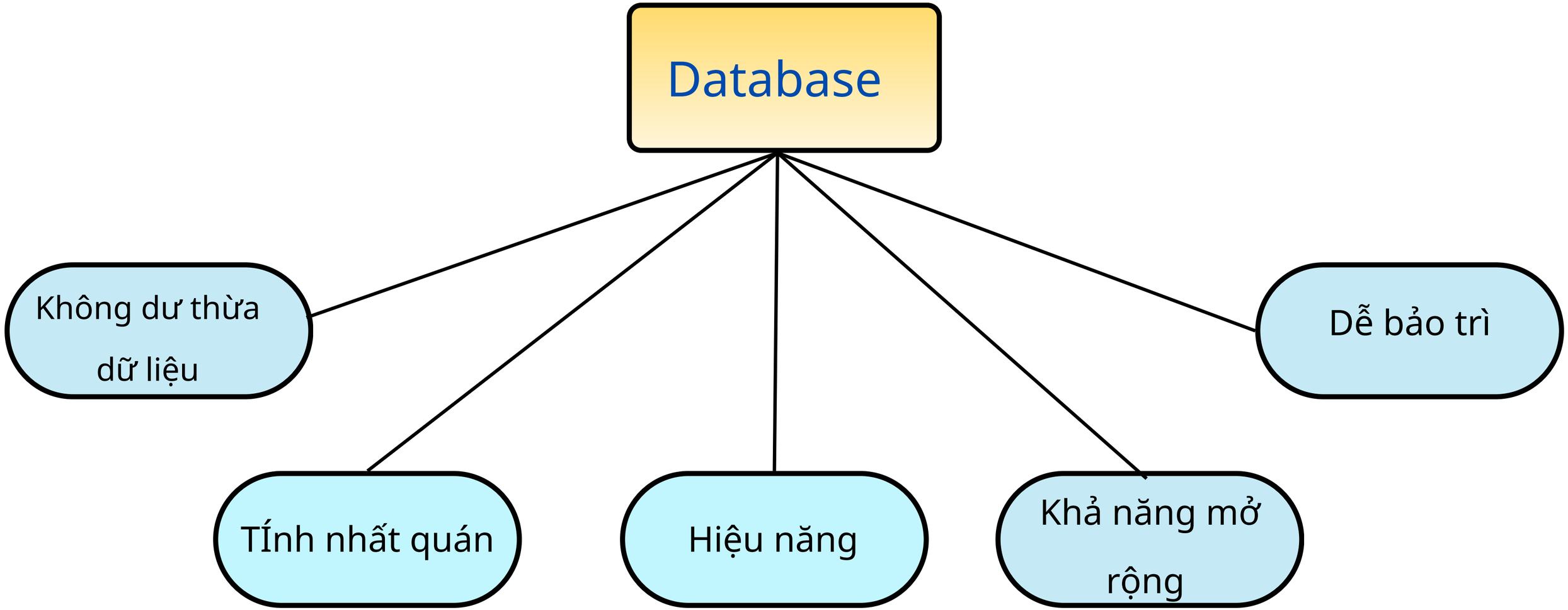
Thuộc tính (Attribute) là các đặc điểm hoặc thông tin mô tả một thực thể.

Mối quan hệ (Relationship) là sự liên kết giữa hai hoặc nhiều thực thể trong hệ thống.

Quy trình các bước

Process Data Modeling là các bước chuyển từ nghiệp vụ thực tế sang mô hình dữ liệu.

1. Phân tích nghiệp vụ (Domain)
2. Xác định Entity
3. Xác định Attribute
4. Xác định Relationship
5. Chuẩn hóa dữ liệu (Normalization)


Database
Không dư thừa dữ liệu
TÍnh nhất quán
Hiệu năng
Khả năng mở rộng
Dễ bảo trì

RESOURCE DESIGN

RESOURCE DESIGN

Quá trình xác định, tổ chức và biểu diễn các tài nguyên (resources) trong hệ thống dưới dạng các URI, đồng thời định nghĩa cách tương tác với chúng thông qua các HTTP methods trong kiến trúc RESTful API.

Tài nguyên

- Một thực thể hoặc dữ liệu trong hệ thống mà có thể được định danh và truy cập thông qua API.

Ví dụ : customers , orders ....

- Mỗi tài nguyên có 1 địa chỉ duy nhất để truy cập định nghĩa bằng URL

Ví dụ : /users/123 , /users .....

Ánh xạ Phương thức HTTP

Resource Design kết nối các hành động của cơ sở dữ liệu (CRUD) với các phương thức HTTP tiêu chuẩn để thao tác trên tài nguyên:

- GET : Đọc
- PUT : Ghi đè
- DELETE : Xóa
- PATCH : Sửa
- POST : Tạo

Biểu diễn tài nguyên

Tài nguyên được phơi bày không nhất thiết phải ánh xạ 1-1 với cấu trúc bảng trong cơ sở dữ liệu. Resource Design quyết định cách dữ liệu được định dạng và trả về cho client.

RESOURCE TREE

RESOURCE TREE

Định nghĩa

Cấu trúc tổ chức các resource trong hệ thống theo dạng phân cấp (hierarchical), thể hiện mối quan hệ giữa các resource.

Vai trò

- Tổ chức resource logic
- Nền tảng thiết kế URL
- Kết nối Data Model – API
- Giúp API dễ hiểu, dễ dùng
- Hỗ trợ mở rộng hệ thống

Đặc điểm

- Cấu trúc phân cấp (hierarchical), có quan hệ parent – child
- Phản ánh quan hệ dữ liệu (1–n, n–n qua resource trung gian)
- Mỗi resource là một node độc lập, có định danh riêng (ID)
- Không tạo resource mới, chỉ tổ chức lại resource đã có
- Có thể ánh xạ trực tiếp sang cấu trúc URL (REST API)

```
Library (root resource)
│
├── Users
│   ├── User Detail (GET /users/<id>)
│   ├── Create User (POST /users)
│   ├── Update User (PUT /users/<id>)
│   └── Delete User (DELETE /users/<id>)
│
├── Books
│   ├── Book Detail (GET /books/<id>)
│   ├── Create Book (POST /books)
│   ├── Update Book (PUT /books/<id>)
│   └── Delete Book (DELETE /books/<id>)
│
└── Loans
    ├── Loan Detail (GET /loans/<id>)
    ├── Create Loan (POST /loans)
    ├── Update Loan (PUT /loans/<id>)
    └── Delete Loan (DELETE /loans/<id>)
```

Nguyên tắc thiết kế Resource Tree

- Xác định resource gốc (Root Resource)

Chọn resource chính làm điểm xuất phát để tổ chức tree.

- Tổ chức parent – child rõ ràng

Resource con gắn trực tiếp với cha, phản ánh quan hệ dữ liệu.

- Mỗi resource là node độc lập:

Mỗi resource có ID riêng, truy cập độc lập.

- Giữ cấu trúc đơn giản:

Tránh nested quá sâu; tối ưu 2–3 cấp để dễ đọc, dễ bảo trì.

- Hỗ trợ mở rộng và bảo trì:

Thêm resource mới mà không phá cấu trúc hiện tại.

Các chiến lược phân trang

Offset / Limit Pagination

Cơ chế hoạt động

Sử dụng offset (vị trí bắt đầu) và limit (số lượng records) để xác định dữ liệu cần lấy

Ưu điểm

- Dễ triển khai và tương thích với hầu hết cơ sở dữ liệu.
- Có thể truy cập trực tiếp bất kỳ trang nào bằng cách thay đổi offset.
- Dễ hiển thị số trang và điều hướng cho người dùng.

Đặc điểm

- Là chiến lược phân trang phổ biến nhất, đơn giản và dễ triển khai.
- Sử dụng offset để bỏ qua số bản ghi đầu và limit để xác định số bản ghi cần lấy.
- Thường phù hợp với dữ liệu nhỏ đến trung bình.

Ví dụ Offset / Limit Pagination

Request

```
GET /books?page=2&size=10
```

SQL

```
SELECT *
FROM books
ORDER BY id
LIMIT 10 OFFSET 20;
```

```python
# Offf limit pagination
def get_books(page=None, size=None):
    books = book_repository.get_all_books()
    if page is not None and size is not None:
        offset = page * size
        books = books[offset : offset + size]
    return books
```

Nhược điểm

- Hiệu năng giảm khi offset lớn (cần scan qua các bản ghi trước).
- Dữ liệu thay đổi trong khi phân trang có thể gây skip hoặc duplicate bản ghi.

Cursor-based Pagination

Cơ chế hoạt động

Client gửi cursor (ID/timestamp) → Server lấy bản ghi lớn hơn cursor, giới hạn limit → Server trả client + cursor mới.

Ưu điểm

- Hiệu năng cao với dữ liệu lớn.
- Không bị bỏ sót hoặc trùng khi dữ liệu thay đổi.
- Phù hợp với streaming hoặc realtime feed.

Đặc điểm

- Sử dụng cursor (thường là ID hoặc timestamp) làm điểm bắt đầu cho trang tiếp theo.
- Thích hợp với dữ liệu lớn hoặc dữ liệu thay đổi liên tục.
- Không phụ thuộc vào offset nên tránh vấn đề skip/duplicate.

Ví dụ Cursor-based Pagination

Request GET /loans?last_id=10&size=5

```
Cursor-based pagination
f get_loans(last_id=None, size=5):
  loans = loan_repository.get_all_loans()
  loans_sorted = sorted(loans, key=lambda x: x.id)  # sắp xếp theo i
  start_index = 0
  if last_id:
      for i, l in enumerate(loans_sorted):
          if l.id == last_id:
              start_index = i + 1
              break
  return {
      "loans": loans_sorted[start_index : start_index + size],
      "next_cursor": (
          loans_sorted[start_index + size - 1].id
          if start_index + size - 1 < len(loans_sorted)
          else None
      ),
  }
```

SQL

```
SELECT *
FROM loans
WHERE id > 10
ORDER BY id
LIMIT 5;
```

Nhược điểm

- Khó triển khai hơn so với Offset/Limit.
- Không dễ nhảy trực tiếp tới một trang bất kỳ.
- Cần xác định trường cursor duy nhất, có thứ tự ổn định.

Page-based Pagination

Cơ chế hoạt động

Client gửi page + size → Server tính offset = (page-1)*size → DB trả limit bản ghi từ offset → Server trả client.

Ưu điểm

- Dễ dùng cho front-end, trực quan với người dùng.
- Dễ hiển thị số trang và điều hướng (next, previous, jump).
- Dễ tích hợp vào UI, ví dụ table, danh sách.

Đặc điểm

- Dựa trên số trang (page number) và kích thước trang (page size).
- Thường là wrapper của Offset/Limit, nhưng dễ sử dụng với front-end.

Ví dụ Page-based Pagination

Request Get /users/?page=0&size=5

```python
# Page-based pagination
def get_users(page=None, size=None):
    users = user_repository.get_all_users()
    total = len(users)
    if page is not None and size is not None:
        offset = page * size
        users = users[offset : offset + size]
    return {"users": users,
            "total": total,
              "page": page,
               "size": size}
```

SQL

```
SELECT *
FROM users
ORDER BY id
LIMIT 5 OFFSET 0;
```

Nhược điểm

- Vẫn chịu nhược điểm của Offset/Limit khi trang lớn (chậm).
- Dữ liệu thay đổi giữa các trang có thể gây skip hoặc duplicate.
- Không tối ưu cho dữ liệu liên tục hoặc realtime.

SO SÁNH

Tiêu chí	Offset / Limit	Cursor	Page - Based
Hiệu suất	Thấp khi dữ liệu lớn	Cao	Trung bình
Ổn định dữ liệu	Không ổn định khi dữ liệu thay đổi	Rất ổn định	Trung bình
MMetadatcho client	Không	chỉ next_cursor	Có
Nhảy trang tùy ý	CÓ	không	Có
Độ phức tạp	Thấp	Cao	Thấp - Trung bình

Cảm ơn mọi người đã lắng nghe